PROJECT REPORT

JARVIS

An Evidence-First Multimodal LLM Investigation Agent

M2LLM - Intelligence and Agent Orchestration

System Requirements · System Implementation · Testing Methodology
Functional and Non-Functional Test Cases · Bugs and Fixes · Testing Tools · Project Schedule · GUI

Repository: github.com/24markjr/M2LLM
Report date: 21 October 2026

Prepared by: _____
Guide: _____

Contents

1. Introduction 4
1.1 Overview 4
1.2 Problem statement 4
1.3 Objectives 4
1.4 Scope 4
2. System Requirements 5
2.1 Functional requirements 5
2.2 Non-functional requirements 6
2.3 Hardware requirements 6
2.4 Software requirements 6
2.5 Constraints and assumptions 7
3. System Implementation 8
3.1 Architecture 8
3.2 The investigation pipeline 8
3.3 Key design decisions 9
3.4 Data model and persistence 9
3.5 API 9
3.6 Web console (Mission Control) 10
3.7 Integrated teammate components 10
3.8 Security 10
3.9 Deployment 11
4. Testing Methodology 12
4.1 Levels of testing 12
4.2 Unit and integration testing 12
4.3 Invariant and adversarial testing 12
4.4 Agent evaluation 12
4.5 Continuous integration 13
4.6 Regression testing 13
5. Functional Test Cases 14
6. Non-Functional Test Cases 16
6.1 Evaluation baseline 16
6.2 Comparison with the plain model (Phase U7) 17
7. Bugs and Fixes 18
7.1 Notable defects 18
7.2 Defect register 18
8. Testing Tools Used 24
9. Project Schedule 25
9.1 Milestones 25

9.2 Remaining work 25
10. Graphical User Interface 27
11. Conclusion 32

Right-click the table above and choose "Update Field" in Word to fill in page numbers.

1. Introduction

1.1 Overview

JARVIS is a locally-run investigation agent built on a large language model (LLM). A user asks a question in plain language - for example, “Do the Aurora project reports contradict each other on the timeline or the finances?” - and attaches the files the answer should come from: reports, spreadsheets, PDFs, Word documents, scanned images, meeting subtitles, audio, video or security logs. JARVIS plans the work, reads every file, draws findings, checks each finding against the exact lines it cites, looks for what is missing, and writes a report that says plainly what it could not establish.

The project’s central idea is that an investigation answer is only as good as its receipts. A plain LLM produces fluent text that cannot be traced to a source line; JARVIS produces findings that each carry citations to a file and a line, a verification verdict from an independent checker, and a confidence value that is computed from the evidence rather than asked of the model. Uncertainty is shown, never smoothed over.

1.2 Problem statement

Investigators, auditors, analysts and security teams routinely reconcile many documents that disagree with each other. General-purpose chat assistants are tempting for this work, but they have three properties that make them unsafe for it: they invent facts with confidence (hallucination), they cannot show which line of which file a statement rests on, and they give no signal when the evidence is insufficient. In addition, sensitive case material often cannot be sent to a cloud service.

1.3 Objectives

- Answer an investigative question across many documents of mixed formats, entirely on local hardware.
- Ground every finding in cited source lines, and verify each finding independently of the reasoning that produced it.
- Detect contradictions, gaps and competing explanations, and state what would change the answer.
- Make the agent’s plan, progress and decisions visible live, and replayable afterwards.
- Keep a chain of custody for evidence, a human review step and an audit log suitable for professional use.
- Measure the agent with a reproducible evaluation harness instead of anecdote, and refuse to compare incomparable runs.

1.4 Scope

This report covers the intelligence and orchestration layer (Member 1) together with the components ported in-process from Members 2, 3 and 4 (retrieval and document formats; the knowledge graph; memory, the trust layer and the injection guard), the web console, and the evaluation and continuous-integration infrastructure. It describes the system as it stands at commit 268e719 on 21 October 2026.

2. System Requirements

Requirements were derived from the project specification and refined phase by phase; each is traced to the component that satisfies it. Functional requirements say what the system does; non-functional requirements say how well, and under what constraints.

2.1 Functional requirements

ID	Requirement	Realised by
FR-01	Accept an investigation objective in natural language, with a mode (Investigate, Compare, Timeline, Security logs)	Home composer; POST /api/v1/missions
FR-02	Ingest text, CSV, PDF (text and scanned), Word, Excel, images, subtitles (SRT), audio, video, JSONL/XML/plain logs	Phase 38-39; app/tools/formats.py, media.py
FR-03	Interpret the objective into a validated intent: goal, required operations, entities	Intent engine (Phase 6)
FR-04	Produce an executable plan as a dependency graph (DAG) of tasks, validated and repaired deterministically	Planner + validator (ADR-006)
FR-05	Execute tasks with registered tools, concurrently where dependencies allow	Execution engine, tool router
FR-06	Detect evidence gaps and re-plan within a bounded number of iterations	Evidence-gap detector, replanning controller
FR-07	Draw findings, each with citations to a document and a line, row, page or character range	Reasoning engine
FR-08	Verify each finding independently against its cited text (model, lexical and specifics checks)	Composite verifier
FR-09	Compute confidence from resolution rate, evidence strength, source agreement and a classification ceiling, and explain it	Confidence decomposition (Phase 46)
FR-10	Classify statements (fact, source claim, inference, contradiction, risk …) and weigh source authority	Phase 45
FR-11	Build a knowledge graph of entities, claims and conflicts; store it in Neo4j; explore it in 2D/3D	Phases 28-32
FR-12	Build timelines and compare dated claims	Knowledge timeline
FR-13	Generate competing explanations and name the best next evidence	Hypotheses (Phase 47)
FR-14	Let a user challenge any finding; search for counter-evidence; keep before and after	Challenge (Phase 48)
FR-15	Human review of findings and an append-only audit log	Phase 50
FR-16	Evidence Locker: SHA-256 identity, integrity checks and chain of custody for every file	Phase 43
FR-17	Organise investigations into cases; rename, pin, search and delete	Phase 44, U1
FR-18	Produce a report with verified, uncertain and rejected findings and limitations; export PDF, DOCX, Markdown, JSON	Phases 18, 51
FR-19	Stream progress live and resume without gaps after a disconnect	SSE with Last-Event-ID (ADR-008)
FR-20	Record runs and replay them in the same interface	Trace replay (Phase 22, U5)
FR-21	Remember findings and entities across investigations	Memory (Phase 33)
FR-22	Detect prompt-injection attempts inside documents and treat them as data	Injection guard (Phase 27)
FR-23	Analyse security logs: entities, IOCs, incident timelines, MITRE ATT&CK mapping and attack paths	Phases 52-53
FR-24	Evidence board: cited text highlighted, investigation pin board of connections, timeline, explanations, report	Phase U6

ID	Requirement	Realised by
FR-25	Verification receipt and side-by-side comparison with the plain model	Phase U7
FR-26	Evaluation harness with datasets, metrics, thresholds and committed reports	Phase 20, 34, 41

2.2 Non-functional requirements

ID	Quality	Requirement
NFR-01	Privacy / locality	All inference runs on a local model through Ollama; no document leaves the machine.
NFR-02	Grounding	Unsupported-claim rate below the 0.15 ceiling; every reported finding has resolved evidence.
NFR-03	Honesty	No hand-written metric anywhere; incomparable evaluation runs are refused; failures are reported, not hidden.
NFR-04	Transparency	Operational decisions are emitted as events; raw model deliberation is never stored or streamed.
NFR-05	Boundedness	Every adaptive loop (replanning, repairs, tool calls) has a positive, configured ceiling.
NFR-06	Determinism	Temperature 0 and a fixed seed for analytical roles; repeated evaluation runs give identical metrics.
NFR-07	Performance	A three-document investigation completes in roughly 30-60 s on a laptop GPU (RTX 4050, 6 GB).
NFR-08	Reliability	A failed tool, model call or store degrades with a recorded reason instead of failing the run.
NFR-09	Security	Path traversal refused; uploads and evidence git-ignored; injection scanning; integrity checks on evidence.
NFR-10	Maintainability	Typed throughout (mypy --strict, TypeScript strict), linted, and documented by ADRs and logs.
NFR-11	Usability and accessibility	Light and dark themes, keyboard navigation, no colour-only meaning, no horizontal scrolling, phone widths.
NFR-12	Portability	Docker Compose for Postgres and Neo4j; Windows, macOS and Linux supported.

2.3 Hardware requirements

Component	Minimum	Used in development
Processor	4-core x86-64	Laptop-class multi-core CPU
Memory	16 GB RAM	16 GB
GPU	Optional; CPU inference works, slowly	NVIDIA RTX 4050, about 6 GB VRAM
Storage	15 GB free (models, Docker images, databases)	SSD
Network	Only for installation; runs offline afterwards	-

2.4 Software requirements

Software	Version	Purpose
Operating system	Windows 11, macOS or Linux	Host (development on Windows 11 Home with WSL2)
Python	3.12	Backend engine and API
Node.js	22 or later	Web console build and tests

Software	Version	Purpose
Docker Desktop	with Compose	PostgreSQL 16 + pgvector, Neo4j 5 Community
Ollama	0.4 or later	Local model server
Models	qwen3:4b, nomic-embed-text, qwen2.5vl:3b (optional)	Reasoning, embeddings (768-d), vision
Browser	Any current Chromium, Firefox or Safari	Mission Control console
Optional media extras	RapidOCR, pypdfium2, faster-whisper	OCR, scanned PDFs, speech-to-text

2.5 Constraints and assumptions

- The model is small (4 billion parameters) so that it fits a laptop; the design compensates with planning, verification and deterministic repairs rather than relying on model size.
- Port 5432 was already used by a native PostgreSQL service on the development machine, so the container is published on 5433.
- Windows 11 Home has no Hyper-V; Docker runs on WSL2.
- GitHub-hosted CI runners have no local model, so CI validates committed evaluation reports instead of running the agent, and says so.

3. System Implementation

3.1 Architecture

JARVIS is a modular monolith (ADR-001): one Python process holds the engine, the API and the integrations, with strict module boundaries rather than network boundaries. This keeps a laptop deployment simple and lets the whole pipeline be tested in-process. Orchestration is custom-written rather than built on an agent framework (ADR-004), so that every step, ceiling and failure mode is explicit and testable.

Layer	Responsibility	Key modules
Web console	React + TypeScript single-page app: home, journal, evidence board, cases, library, insights, activity	frontend/src
HTTP API	FastAPI routes, error contract with stable error codes, SSE streaming, mission registry	app/api
Orchestration	One headless pipeline shared by the CLI, the API and replay	app/orchestration/mission.py
Intelligence	Intent, planner, router, execution, reasoning, verification, gaps, replanning, synthesis, hypotheses, challenge, confidence, trust, proof	app/intelligence
Tools	Document loading, format parsing, media understanding, knowledge tools	app/tools
LLM abstraction	Provider protocol, Ollama and deterministic echo providers, structured output with repair	app/llm
Persistence	PostgreSQL via async SQLAlchemy and Alembic; pgvector; Neo4j graph store; run archives	app/database, app/integrations
Cross-cutting	Typed configuration, event bus, structured logging, security	app/core, app/security

3.2 The investigation pipeline

Every investigation passes through the same stages, each of which emits typed events to the event bus. The console, the database sink and the recorder are subscribers to that bus, so what the user sees is exactly what the engine did.

1. Understanding - the intent engine turns the question into a goal, the required operations (made mandatory by the chosen mode) and the entities involved.
2. Planning - the planner proposes a task graph; the validator checks it is a legal DAG that covers the required operations and repairs it deterministically when it is not. Repairs are recorded, never hidden.
3. Executing - tasks run in dependency waves; the router selects a registered tool for each; observations are collected with their locators.
4. Reasoning - candidate findings are drawn from the observations, each with citations; a relevance gate drops claims that do not answer the question, emitting the reason.
5. Verifying - a composite verifier checks each finding against only its cited text: a model check, Member 4’s lexical rule and a specifics check for dates and figures.
6. Gap detection and replanning - unresolved elements become evidence gaps; the replanning controller inserts tasks to close them, within a bounded number of iterations.

7. Synthesis - the report is assembled: facts, counts, confidences and citations are injected programmatically; the model writes prose only inside fixed sections, and every narrative sentence is checked again.

3.3 Key design decisions

ADR	Decision	Reason
001	Modular monolith	Laptop deployment; in-process testing of the whole pipeline
002	PostgreSQL	Relational integrity for missions, tasks, findings, events, cases and audit
003	Ollama for local models	Privacy and offline use; GPU acceleration on Windows natively
004	Custom orchestration	Explicit, bounded, testable control flow instead of a framework
005	pgvector	Embeddings in the same database as the structured data
006	Task graph (DAG)	Parallel execution and dependency correctness as a measurable property
008	Server-Sent Events	One-way live stream with gap-free resume via Last-Event-ID
009	Port teammates in-process	One system to test and demo, rather than four services to keep alive
010	Neo4j knowledge graph	Graph queries over entities, claims and conflicts
011	3D knowledge explorer	Investigators see how evidence connects
012	Seeing and hearing	OCR, vision model and speech-to-text make images, scans, audio and video readable
013	Stored retrieval	Workspace-scoped search by meaning over ingested documents
014	Evidence Locker	SHA-256 identity and chain of custody for every file
015	Statement types and authority	A contradiction weighs who said each side
016	Review and audit	A human decision on findings, with an append-only log
017	Report writer	PDF and Word exports generated from the same model as the console
018	ATT&CK mapping	Security-log findings mapped to MITRE techniques with their evidence lines

3.4 Data model and persistence

PostgreSQL 16 with the pgvector extension holds missions, tasks, findings, events, document chunks with 768-dimensional embeddings, workspaces, memory (episodic and semantic), evidence items and their custody, cases, reviews and the audit log. The schema is versioned by Alembic migrations; the newest migration is round-tripped (down and up) in CI so a rollback is known to work. Neo4j 5 holds the knowledge graph; when it is unavailable the system falls back to an in-memory store and logs one degraded line. Finished runs are additionally archived to disk so they survive a restart, and recordings capture every event plus the final state for replay.

3.5 API

The FastAPI application exposes 71 documented paths (docs/openapi.json, checked for freshness in CI). Every failure carries a stable error_code the frontend branches on, never prose. Main groups:

Group	Examples
Missions	POST /missions, GET /missions/{id}, /tasks, /plan, /events, /stream, /findings, /gaps, /report, /hypotheses, /challenges, /sources, /receipt, /comparison
Knowledge	/missions/{id}/knowledge: entities, relationships, claims, conflicts, timeline, search, graph, nodes

Group	Examples
Evidence and cases	/evidence (list, detail, verify, download), /cases (open, update, pin, search, delete)
Review and audit	/missions/{id}/reviews, /audit
Documents and context	/documents (upload, formats), /context (workspaces, retrieval)
Memory	/memory, episodes, facts, entities, forget a run
Evaluation and replay	/evaluation (reports, quality, directions), /recordings
Reports and cyber	/investigation-report and its exports, /cyber, /attack

3.6 Web console (Mission Control)

The console is React 19 with TypeScript in strict mode and Vite, with hand-written CSS on a token-based design system (light by default, dark in Settings) and no UI framework: each omitted library is a toolchain that cannot break during a demonstration. It is organised as an application shell (sidebar with cases grouped by date, Ctrl+K search, top bar) around these pages:

- **Home and composer:** “What do you need to find out?”, files by upload, drop, library or paste, modes, domain packs with starter questions, and a “Needs your attention” strip.
- **Investigation journal:** the question, seven plain-language steps while it runs, the answer with each sentence marked verified, inferred, not established or unknown, finding cards with numbered citations, review and challenge, and three closing sections: what could not be established, what would change the answer, and the best next evidence.
- **Evidence board:** beside the journal - Sources (the text each file was read as, with every cited span highlighted), the investigation Pin board (findings and files joined only by connections the run holds), Timeline, Explanations, Proof (verification receipt and plain-model comparison) and Report (exports, knowledge graph, attack path).
- **Knowledge explorer:** the run’s entities, claims and documents in 3D (2D under reduced motion), with trails from a finding to its evidence.
- **Library, Insights and Activity:** evidence and documents; evaluation quality, memory and recordings; the audit log.

3.7 Integrated teammate components

Member	Contribution	Where it lives now
Member 2	Workspace retrieval by meaning; document formats (Word, Excel, images)	app/intelligence/context, app/tools/formats.py
Member 3	Knowledge extraction, entity resolution, contradictions, timeline, graph	app/intelligence/knowledge, Neo4j store
Member 4	Episodic, semantic and working memory; lexical trust evaluator; injection guard; benchmark	app/memory, app/intelligence/trust, app/security

3.8 Security

- Documents are scanned for prompt-injection patterns; a flagged document is still read, strictly as data, and the flag is shown on the mission.

- Every evidence file is identified by its SHA-256, checked for integrity before use, and excluded with a stated reason if it fails.
- File names are resolved strictly inside their folder, refusing path traversal; uploads, evidence and run archives are git-ignored.
- Bandit-style lint rules (ruff S) and blind-except rules (BLE) run on every change.

3.9 Deployment

Locally, scripts/start-jarvis.ps1 starts Postgres and Neo4j in Docker, checks Ollama, and opens the API (port 8000) and the console (port 5173). For sharing, a GitHub Actions workflow builds a static preview of the console and publishes it to GitHub Pages (24markjr.github.io/M2LLM); with no engine behind it, the preview replays committed recordings and says that live investigations run locally.

4. Testing Methodology

Testing follows one rule: a check that can fail cheaply runs first, and a check that cannot fail is not a check. The strategy combines a conventional test pyramid with two layers specific to an LLM agent - invariant and adversarial suites that encode what the agent must never do, and an evaluation harness that measures the agent on scenarios with known answers.

4.1 Levels of testing

Level	What it covers	Scale
Unit tests	Every engine, schema, tool and route in isolation, with a deterministic echo model	1,282 tests (pytest)
Integration tests	Real PostgreSQL and Neo4j: persistence, cases, evidence locker, memory, review store, context store	53 tests
Frontend tests	Route parsing, journal logic, board highlighting and pin-board connections, proof wording, themes, components	117 tests (Vitest)
Invariant suite	The specification's "must not do" list, as tests that fail if violated	Part of the unit suite
Adversarial suite	Injection, malformed plans, corrupt input, zero-finding runs, tool storms	Part of the unit suite
Agent evaluation	Ten metrics over fourteen scenarios with known answers, against the real local model	Committed reports
Specialised benchmarks	Trust benchmark, injection cases, challenge benchmark, ATT&CK mapping benchmark	Re-run in CI
End-to-end UI checks	Live runs driven through the console in headless Chrome, at desktop and phone widths	Per UI phase

4.2 Unit and integration testing

Unit tests run with a deterministic echo provider so that they are fast and repeatable, and pytest-randomly reorders them on every run so that hidden state leaking between tests is caught (the seed is printed for reproduction). Integration tests run against real PostgreSQL with pgvector and real Neo4j; in CI they are configured to fail rather than skip if a database did not come up, because a silently skipped suite is "a green tick over nothing".

4.3 Invariant and adversarial testing

The agent's invariants are written as tests: every adaptive loop is bounded; model deliberation is never recorded; a finding without resolved evidence is never reported as verified; a zero-finding run produces an honest report rather than an invented one; prompt-injected instructions inside a document are not followed. Adversarial tests feed malformed plans, corrupt files, tool storms and injection attempts and assert the system degrades with a recorded reason.

4.4 Agent evaluation

The evaluation harness (python -m app.cli eval --suite all) runs the real agent on fourteen scenarios - including four negative cases whose correct answer is "nothing found" and one planted prompt injection - across four document families and seven file formats. Each report is stamped with the model, prompt versions and a configuration hash; two reports with different stamps are refused as incomparable rather than quietly

compared. Thresholds on the metrics are enforced, and the baseline was run three times with identical metrics apart from latency.

4.5 Continuous integration

GitHub Actions runs on every push and pull request, in a deliberate order:

8. Lint (ruff check and ruff format --check).
9. Types (mypy --strict over the whole backend).
10. Unit, invariant and adversarial tests with a coverage floor on the intelligence and schema packages.
11. Integration tests against PostgreSQL (pgvector) and Neo4j service containers, plus a migration round-trip.
12. OpenAPI contract freshness - the frontend's types come from it.
13. Frontend: TypeScript check, production build and Vitest.
14. Evaluation reports: committed reports validated; trust, security, challenge and ATT&CK benchmarks re-run and compared; documented figures checked against the baseline.

4.6 Regression testing

Every defect found by running the system is first reproduced by a test, then fixed; the test stays. Documented metrics are machine-checked against the committed baseline so that a re-run cannot leave the README confidently wrong.

5. Functional Test Cases

Each functional test case is automated (unit, integration or frontend suite) unless it names a live run, in which case it was executed against the local model and inspected in the console. The defect numbers refer to Section 7.

ID	Test	Input	Expected result	Result
TC-F01	Start an investigation	Question + 3 Aurora files, mode Investigate	Mission accepted (202), run id returned, stream begins	Pass
TC-F02	Intent extraction	“identify contradictions between timeline and finances”	Goal and required operations include contradiction detection	Pass
TC-F03	Mode makes operations required	Mode Timeline	Planner must cover the timeline operations	Pass
TC-F04	Plan validation and repair	Plan missing a terminal task	Deterministic repair recorded; plan executes	Pass
TC-F05	Plan missing a required operation	Planner omits an operation	Plan completed by repair, mission not failed (BUG-035)	Pass
TC-F06	Cross-document contradiction	Aurora reports (30 April vs 14 May; 380,000 vs 450,000)	Two contradiction findings, both verified, all citations resolved	Pass
TC-F07	Negative case	Consistent reports	No findings; report states nothing was established	Pass
TC-F08	Composite verification	Claim with a date absent from its cited line	Rejected by the specifics check	Pass
TC-F09	ID mistaken for a date	“delivered on 9012” (shipment number)	Flagged, not verified (BUG-021)	Pass
TC-F10	Citation with spaces in file name	“2. Detailed Project Timeline.pdf”	Citation parses and resolves (BUG-036)	Pass
TC-F11	Evidence gap and replanning	Finding element unresolved	Gap detected, tasks inserted, iteration bounded	Pass
TC-F12	Relevance gate	Background restatements	Dropped with FINDING_DISCARDED and a reason	Pass
TC-F13	Scanned image	Delivery-note PNG	Text read by OCR; findings cite it	Pass
TC-F14	Excel and Word	orion_ledger.xlsx, orion_minutes.docx	Rows and paragraphs parsed with locators	Pass
TC-F15	Subtitles	orion_meeting.srt	Timed lines read and cited	Pass
TC-F16	Security logs	Lateral-movement JSONL, Windows XML	Techniques mapped to ATT&CK with evidence lines	Pass
TC-F17	Benign logs	Ordinary admin activity	No attack asserted (BUG-038)	Pass
TC-F18	Prompt injection	Document containing instructions to the model	Flagged; instructions not followed; read as data	Pass
TC-F19	Live stream and resume	Disconnect after event 3	Reconnect with Last-Event-ID delivers exactly the missed events	Pass
TC-F20	Knowledge graph	Finished mission	Entities, claims and conflicts stored in Neo4j; explorer opens	Pass
TC-F21	Hypotheses	Contradiction scenario	Competing explanations with support, against and next evidence	Pass
TC-F22	Challenge a finding	Challenge F-001	Counter-evidence searched; outcome recorded; finding unchanged	Pass
TC-F23	Human review and audit	Accept a finding	Review stored; audit entry written	Pass

ID	Test	Input	Expected result	Result
TC-F24	Evidence Locker	Upload a file	SHA-256 identity, custody events, integrity check	Pass
TC-F25	Cases	Create, rename, pin, search, delete	Each action persisted and audited; custody kept on delete	Pass
TC-F26	Report export	PDF, DOCX, MD, JSON	Files generated from the same report model	Pass
TC-F27	Replay	Committed recording	Plays in the journal at up to 20x; says it is a replay	Pass
TC-F28	Run survives restart	Restart the API	Finished run restored from its archive; all routes answer	Pass
TC-F29	Evidence board viewer	Click a citation	File opens with the cited span highlighted and scrolled into view	Pass
TC-F30	Investigation pin board	Two findings citing the same file	Strings only for connections the run holds; reason listed	Pass
TC-F31	Verification receipt	Finished run	Receipt with file hashes and a stable self-hash that changes if a file changes	Pass
TC-F32	Plain-model comparison	Ask the plain model	Both answers scored by the same sentence checker; invented figures listed	Pass
TC-F33	Engine not running	Open the console with the API stopped	Page states the engine is not running and how to start it	Pass

6. Non-Functional Test Cases

Measured values come from the committed evaluation baseline 20261006T095620-qwen3-4b-all, run three times, and from the test and CI runs of 20-21 October 2026.

ID	Quality	Method	Criterion	Measured	Result
TC-N01	Grounding	Evaluation suite, 14 scenarios	Unsupported-claim rate ≤ 0.15	0.000	Pass
TC-N02	Evidence coverage	Evaluation suite	Every finding has resolved evidence	1.000	Pass
TC-N03	Verification	Evaluation suite	Findings surviving the independent check	0.976	Pass
TC-N04	Tool selection	Evaluation suite	Correct tool for each task	1.000	Pass
TC-N05	Determinism	Baseline run three times	Identical metrics across runs	Identical except latency	Pass
TC-N06	Performance	Baseline, laptop GPU	Mean latency per scenario	32.8 - 37.1 s	Pass
TC-N07	Boundedness	Configuration test	Every adaptive loop has a positive ceiling	All bounded	Pass
TC-N08	Transparency	Event inspection	No model deliberation stored or streamed	None found	Pass
TC-N09	Security	Injection suite	Injected instructions never followed	All cases pass	Pass
TC-N10	Security	Path traversal on recordings/evidence	Refused	Refused	Pass
TC-N11	Reliability	Neo4j stopped	In-memory fallback, one degraded log line	As expected	Pass
TC-N12	Reliability	Vision call fails during a video	Other frames and speech kept (BUG-026)	As expected	Pass
TC-N13	Maintainability	mypy --strict, ruff	No type or lint errors	Clean, 176 modules	Pass
TC-N14	Contract	OpenAPI export --check	Committed spec matches the API	Up to date	Pass
TC-N15	Usability	Headless Chrome at 1440, 800 and 600 px	No horizontal page scrolling; readable at phone width	As expected	Pass
TC-N16	Accessibility	Keyboard and reduced motion	Tabs, dialogs and pins operable by keyboard; 2D graph under reduced motion	As expected	Pass
TC-N17	Portability	Fresh clone, Docker Compose	Stack starts with one command	As expected	Pass
TC-N18	Recoverability	API restart mid-session	Finished investigations restored from archives	As expected	Pass

6.1 Evaluation baseline

Metric	Value	Meaning
evidence_coverage	1.000	Every finding has resolved evidence
unsupported_claim_rate	0.000	The hallucination proxy
tool_selection_accuracy	1.000	Correct tool chosen for each task
dependency_correctness	0.878	Plan dependencies match the reference

Metric	Value	Meaning
replanning_success	1.000	Gaps closed within the ceiling (vacuous where no gap arose)
intent_accuracy	0.564	Intent matches the reference interpretation
task_efficiency	1.731	Tasks ÷ minimal sufficient tasks (lower is better)
plan_validity	0.714	Plans passing with zero repairs
verification_success	0.976	Findings that survive the independent check

Two figures deserve their explanation. plan_validity of 0.714 means four plans in fourteen still need a deterministic repair before they execute; the metric records this instead of hiding it. verification_success was 1.000 when a weaker verifier approved an invented finding; the composite verifier rejected it and the figure fell, which is the honest direction. A verifier that approves everything scores 1.000 and is worthless.

6.2 Comparison with the plain model (Phase U7)

On a live Aurora run with qwen3:4b, the same question and files were given to the same model in one plain prompt. Both answers were checked sentence by sentence against every line of the files by the same checker. The plain model had 5 of 7 sentences supported by a single line and cited nothing checkable; JARVIS reported two verified contradiction findings with all five citations resolved, but its three summary sentences were not credited, because the checker matches a sentence to one line and therefore credits restatements, not conclusions drawn across two documents. This limitation is reported as measured; an independent cross-document checker is planned (Section 9).

7. Bugs and Fixes

Forty-one defects were logged in .claude/logs/bug-log.md, every one found by running the system rather than by reading it, and every one fixed with a regression test. By severity: 1 critical, 25 high, 12 medium, 3 low. The most instructive are described first; the complete register follows.

7.1 Notable defects

- **BUG-005 - confabulation on a negative case.** The agent reported eight findings on a scenario whose correct answer is none. Fixed by stating in the reasoning prompt that an empty result is a correct answer and adding a relevance gate that judges each candidate against the objective, emitting a reason for every drop.
- **BUG-006 - one number used as two budgets.** Reasoning bounded its input by the model's output limit, compressing evidence five times more than intended; a scenario with two planted contradictions returned nothing. Input and output budgets were separated.
- **BUG-010 - an unplannable intent.** A defined-but-never-set optional flag made the intent demand 17 operations, so no plan could cover them. Fixed, and the plan size fell from 16 tasks to 7 while findings rose from 0 to 4.
- **BUG-018 and BUG-021 - the verifier passing wrong claims.** A model verifier approved a date its evidence did not contain, and accepted a shipment number as a date. Fixed by the composite verifier with a specifics check.
- **BUG-029 - the wrong text behind every verdict.** The text at a page citation was whichever tool cited it last, so verification since Phase 15 had checked the wrong text. Fixed by keying citation text per tool and document.
- **BUG-036 - real file names never resolved.** The citation pattern disallowed spaces, so every citation of the owner's real documents failed while the test corpora passed. Fixed, and a lesson recorded: test corpora share their authors' naming habits.

7.2 Defect register

ID	Defect	Severity	Cause	Fix
BUG-001	Healthcheck crashed rendering its own report on the Windows console	Low	The report renderer used → for hints and — in two detail strings. The Windows console defaults to code page 1252, which has no mapping for either. The failure happened *while printing a FAIL row*, so the first real failure the script detected was also the first thing that broke it.	Operator-facing output is ASCII-only. → became ->, em dashes became hyphens. Non-ASCII remains fine in comments and docstrings, which are never written to the console.
BUG-002	extract_json returned a fragment for JSON array responses	High	The balanced-delimiter scan tried {/} before [/]. In an array of objects the first { appears at index 1, so the scan locked onto the first element and returned it.	Start the scan from whichever of { or [appears first in the text.
BUG-003	Empty model responses: qwen3 spends its token budget on deliberation	High	qwen3:4b is a reasoning model. Ollama returns the deliberation in a separate thinking field and the answer in response. With a modest num_predict, deliberation consumed the entire budget: The repair loop then dutifully retried three times against a model that	Send think: false on every request, configurable per role via models.yaml. The provider reads only response and never reads thinking.

ID	Defect	Severity	Cause	Fix
			was always going to run out of tokens before answering —…	
BUG-004	Every Aurora citation came back UNRESOLVED	High	The model appends the quoted line after the reference: The parser used rpartition(":"), which split on the last colon and read the quoted prose as the row position. int("...") failed, the locator was rejected, and the whole citation was discarded.	An anchored regex that reads the reference at the start and ignores whatever follows: This accepts the reference the model actually meant without accepting one it did not make.
BUG-005	The agent confabulated on a scenario with nothing to find	High	Two layers, neither at fault on its own: The reasoning prompt never said that returning nothing was acceptable. A model asked for findings produces findings. Nothing in the pipeline asked whether a claim answered the objective. Verification checks whether the evidence supports the claim — and for a restatement, it…	Reasoning prompt v2 states explicitly that an empty result is a correct answer, plus a new relevance gate (prompts/relevance.md, ReasoningEngine._filter_irrelevant) that judges each candidate against the objective before binding. It fails open on any provider error and emits FINDING_DISCARDED with…
BUG-006	Input and output token budgets were the same number	High	compact() bounds how many tokens of observations enter the reasoning prompt. Its own default is Reasoning passed: max_tokens is the cap on what the model may generate. So observations were squeezed five times tighter than intended — and summarising is precisely what strips the dates and figures a contradiction rests…	agent.yaml: reasoning.observation_budget_tokens (6000), carried on AgentBounds. A test asserts the observation budget exceeds the generation budget and never drops below compact()'s own default.
BUG-007	Mission list order varied between identical requests	Low	Sorted by created_at. Two missions created in the same millisecond tie, and Python's sort is stable with respect to dictionary order, which is insertion order — but the reverse sort made the tie resolve unpredictably relative to what a reader expected.	MissionRecord carries a monotonic sequence from itertools.count(), and the listing sorts on that.
BUG-008	mypy called live code dead because finished was a property	Low	MissionRecord.finished was a property. mypy narrows a property's truthiness, so after the early if record.finished: return it treated the value as permanently False and every later check as dead code.	is_finished() — a method. Method calls are not narrowed, and it reads better for mutable state.
BUG-009	One word made an objective unplannable	High	The closed vocabulary has detect_inconsistencies. The model proposed detect_contradictions persistently. The task was dropped as an invented type, which left the operation uncovered, which failed the plan.	_TASK_TYPE_SYNONYMS in the planner — small and explicit, mirroring _SYNONYMS in the intent engine and for the same stated reason: never fuzzy, because a fuzzy matcher reintroduces the silent nearest-match problem a closed vocabulary exists to prevent.
BUG-010	The intent demanded 17 operations, so no plan could cover them	High	Two independent causes: RequiredOperation.optional was dead. The field existed and Intent.mandatory_operations filtered on it, but nothing ever set it. So every operation became one the plan was required to contain — including verify_findings and summarize, which are *pipeline stages* no planner schedules, and…	SUPPORTING_OPERATIONS in schemas/intent.py classifies pipeline-inherent operations in code — which operations are inherent is a fact about this architecture, not a per-run judgement for a model. Plus intent prompt v2: ask for the fewest, state that three to six is normal, and name the specific…
BUG-011	eval crashed before it could compare	Medium	_TOLERANCES was defined at the bottom of cli.py, below if __name__ == "__main__": sys.exit(main()). Under python -m app.cli, main() runs during	Moved above the entry point.

ID	Defect	Severity	Cause	Fix
	against a baseline		module execution, so the assignment below it had not been reached.	
BUG-012	My own prompt example taught the model to write uncheckable citations	Medium	That sentence is verbatim from reasoning.md v2, where I had written it as an illustration of how to phrase a contradiction: state the conflict in a single claim and cite both sides: "Document A gives 30 April, document B gives 14 May" is one finding, not two The model copied the example including the placeholder…	Prompt v3: the example now uses real filenames, and the rule says explicitly never to write "Document A" or "the first document", because a placeholder cannot be checked against anything.
BUG-013	Tasks inserted by replanning were invisible everywhere	High	Two independent halves, which is why it survived a phase that was specifically about showing it.	run_mission now carries the executed graph, and _insert_tasks stamps each task with the revision that created it. A test asserts every inserted task carries both the revision and the gap it was created for.
BUG-014	Two of my own prompts disagreed about what a finding is	Medium	The prompts contradicted each other. reasoning.md said returning no findings is a correct answer; relevance.md said *"keep a negative answer too: 'the two figures agree' answers 'do these conflict?'"*. The model followed the second.	Both prompts now say it: a claim asserting an absence is not a finding, and when the answer is "nothing" the output is an empty list rather than one claim announcing it. A negative conclusion belongs in the report narrative, which is written from the fact that nothing was established. Reasoning v4,…
BUG-015	Verification never saw the line a finding cited	High	Two defects in one function.	evidence_from_observations(observations, documents, page_starts) reads the text at each locator from the documents themselves: the line for :rN; for :pN the lines a tool matched on that page, else the page text, both capped at 1,200 characters. Page locators stay pages. The replanning controller…
BUG-016	A locator's detail could come from another document	Medium	_detail_for attaches the extracted value to each locator in the reasoning prompt, so the model sees report.txt:r8 30 April 2026 rather than a bare reference. It matched items on line only. When one observation carries items from two documents with something on the same line number, finance.txt:r8…	Match document_id as well as line. Items without a document_id still match on line alone, so existing tool output is unaffected. Two tests.
BUG-017	An evaluation report did not record which verifier ran	Medium	A report's comparable_key is model + prompt versions + config hash, and the regression check refuses to compare two reports whose keys differ. The config hash covered temperature, seed, iteration and parallelism ceilings, planning policy and the repair budget. It did not cover…	The hash includes verification_provider and the lexical thresholds. Every report written after this has a different config hash from every report before it, including runs on baseline. That is correct: the stamp now describes something it did not describe before.
BUG-018	The model verifier approved a claim whose decisive date its evidence does not contain	Medium	Not a code defect. The model verifier (qwen3:4b, verification prompt v1) is lenient when it reads real text. BUG-015 hid this: before the fix it read tool summaries and rejected findings for the wrong reason, which kept verification_success at a plausible-looking 0.750.	Fixed. The composite verifier's specifics check (A15) catches it, and became the default on 2026-10-05 by Experiment 004
BUG-019	The negative case confabulates again, and the code did not change	High	aurora_no_contradiction, whose correct answer is no findings, produced an invented finding stating a value that no cited line contains, with no change to the code since the case last passed.	Fixed 2026-10-05: the reasoning engine now discards a candidate whose dates and figures appear in none of its cited lines (unevidenced_values), so the invented finding is no longer produced.

ID	Defect	Severity	Cause	Fix
BUG-020	For a PDF, the reasoning model saw page references and no content	High	The reasoning prompt attaches the extracted values to each citation (_detail_for), so the model reads report.txt:r12 15 January 2026 rather than a bare reference. It parsed only line citations (:rN). Tools cite a PDF by page (:pN), so for a PDF it attached nothing. Printed for real tool output on…	A page citation carries the values the tool found on that page, up to eight, located with the document's page boundaries. The replanning controller already held these since BUG-015 and now passes them to the reasoning engine. After the fix the same probe prints aurora_project_report.pdf:p1 20 April…
BUG-021	The verifier passed "delivered on 9012": a shipment number taken for a date	High	The scenario asks when Shipment 9012 was delivered and by whom. shipment_delivery_confirmation.txt says "delivered to XYZ Traders on 20 September by driver Priya Menon". The run produced a finding "delivered on 9012" - the shipment number taken for a date - which the verifier passed.	Fixed 2026-10-05: the specifics check (numbers_as_dates, composite rule 4) flags a number standing where a date belongs, and the finding is no longer verified.
BUG-022	An 11-scenario run was compared with an 8-scenario baseline, and "passed"	Medium	comparable_key covered the model, the prompt versions and the config hash. The set of scenarios is not configuration, so adding three to the dataset left the key unchanged.	The key includes the sorted scenario ids, and the refusal says so ("... or set of scenarios"). Test: test_reports_over_different_scenarios_are_not_compared. Old reports need no migration: the ids are read from each report's own scenarios.
BUG-023	No mission could use an uploaded file	High	POST /documents stores a file in .agent/uploads/ and says, in its own docstring, that uploads "are then referenced by name when a mission is created, exactly like a fixture". Mission creation resolves names with resolve_document, which looked in .agent/fixtures/ only. An uploaded file's name…	resolve_document also searches .agent/uploads/. test_an_uploaded_file_can_be_used_by_name uploads a Word file and loads it by name; test_api_documents.py (new) covers the endpoint.
BUG-024	Uploaded files were not git-ignored	Medium	.gitignore ignored local traces and local reports, not .agent/uploads/. Nothing had been uploaded on this machine, so nothing leaked; with Phase 38's "Add files" it would have been one git add -A away.	.agent/uploads/ is ignored, with the reason beside it.
BUG-025	Speech recognition failed on every file: faster-whisper and PyAV 19 disagree	High	TypeError: open() got an unexpected keyword argument 'metadata_errors' from inside faster-whisper 1.2.1's own audio decoder, which passes PyAV an option PyAV 19 removed.	Our code decodes the audio with PyAV (16 kHz, mono, float32, bounded) and hands Whisper the samples, which works with any PyAV. test_audio_is_decoded_to_16khz_mono_for_speech_recognition covers the decoder. After the fix: a 7-second spoken sentence transcribed in 2.4 s.
BUG-026	One failed vision call discarded everything read from a video	High	With the vision prompt unavailable, a video's understanding recorded read=1 and kept no lines: the exception escaped the frame loop before the timeline was assembled.	see() treats any failure as "nothing seen" and logs it; what was read and heard is kept. test_a_failing_vision_model_never_loses_what_was_read holds it.
BUG-027	A value taken from a line about something else made a contradiction	High	"The Aurora project report states two different completion dates: 30 April 2026 and 20 April 2026", citing Date: 20 April 2026 - the report's own date. "The written minutes state [the handover] as 1 April 2026", citing the minutes'	For a comparative objective, a value is discarded when two things hold: the cited line holding it shares no content word with the claim (words cut to six letters, generic words like "date" and month names ignored, identifiers kept; table rows exempt), and its own document has a line that does share…

ID	Defect	Severity	Cause	Fix
			installation line. Both values were on the cited lines, so…	
BUG-028	A spreadsheet row's date and column separator read as one amount	High	In Cooling units (36),Polar Systems,2026-03-19,162000 the amount pattern, which took any run of digits and commas, read 19,162000; the reasoning model was handed $19,162,000.	Commas only in groups of three; date spans blanked before amounts are matched (so a year is no longer an amount either).
BUG-029	The text at a page citation was whichever tool cited it last	High	"The recorded spend exceeded the approved budget by $37,500" cited the memo and orion_ledger.xlsx:p1. A page citation carries the lines a tool matched there; the map from citation to text was a plain dict, so the date extraction's rows replaced the budget extraction's "Total spend" row, and the…	The text for a citation merges every observation that cited it. Also new in the same work: a difference of two cited figures counts as grounded (37,500 = 287,500 - 250,000), since the objective asked for exactly that comparison.
BUG-030	A question asking for risks as well as contradictions could only report contradictions	High	The objective asked for contradictions *and* cybersecurity risks, implementation gaps and missing evidence. Because it asked for contradictions, the comparative rule discarded every claim resting on one source - including the kind that answers "risks" ("the SoW specifies no penetration testing").…	When the objective also asks for an assessment (assess_impact or summarize among the operations it asks for, optional or not), a single-source claim is kept and labelled SINGLE SOURCE (Finding.single_source, in the report and the console). A question that only compares is unchanged, which is what…
BUG-031	A contradiction in the knowledge graph could be missing from the report	Medium	The knowledge pass hands conflicting claim pairs to reasoning, but a contradiction became a finding only if the reasoning model restated it, and a 4B model often does not (BUG-005/012).	After reasoning, each contradiction that is reportable - dates or numbers, across two documents, not a document's own date or version, not already covered by a finding that cites two of its sides, for a question that asks for a comparison - becomes a finding built from its two cited lines, without…
BUG-032	A report with no findings did not say why	Medium	No findings, Confidence "None.", and a Limitations section printing the placeholder "(generated from run facts - see the limitations list)". Nothing said that 179 of 191 sections were never read, that five of seven searches found nothing, or that all 22 candidate findings had been discarded before…	Reasoning counts what it discards, by reason. The report lists its real limitations - sections not read, sections that failed, claims not found on their line, searches that found nothing, candidates discarded and why - states what was examined in the Investigation Scope, and an empty Confidence…
BUG-033	Every difference between two claims was reported as a contradiction	High	All six "conflicts" were one document (AURORA_Annexure-1_PS-71.pdf) describing the system in other words on different pages: "description = Automated Unified Reasoning and Optimization Architecture" against "Automated Compliance Validation"; "synchronization_method = continuously reconciles CAD…	Disagreements are split. Contradictions (conflicts): dates and numbers that differ, and short categorical text (four words at most: a name, a status) from different documents. Differing descriptions (differences, new): every other text difference - stored, shown in grey in the graph and the side…
BUG-034	The knowledge pass read the first document and little else	High	The pass reads at most knowledge.max_chunks sections (twelve: one model call each) and took them in order, so the first document filled the budget: 12 of 191 sections read, nearly all from one PDF. It also extracted contact details ("ARCAISYS.phone = 9137815600") as if they were evidence.	The budget is dealt round-robin across documents (every document read before any is read twice); within each document the sections with the most words of the objective - plus a half point each for a date and a figure - come first; what is read is put back in document order. Phone numbers, e-mail…

ID	Defect	Severity	Cause	Fix
BUG-035	A planner that kept leaving out an operation failed the mission before it read a line	High	The planner's only answer to an uncovered operation was to re-prompt, and then fail. The other deterministic repairs (a missing report task, an orphan) existed because re-prompting did not reliably fix those either; coverage had no such repair.	After the last re-prompt, and only when coverage is the plan's *only* fault, a task is added for each missing operation. Which task type implements an operation is fixed by the catalogue (TASK_SATISFIES), so no judgement is involved: the new task reads what the report task was going to read, and…
BUG-036	A citation of a document with a space in its name never resolved	Critical	The locator pattern took the document name as [^\s:]+? - no spaces. Every evaluation corpus uses names like project_report.txt, so the suites never saw it; the owner's documents are named "2. Detailed Project Timeline.pdf" and "Annexure-I-SoW_iDEX_71 (1).docx". Every citation of them failed to…	A document name runs to the first colon followed by a position, spaces included; anything after the position (a quoted knowledge line, which the model copies) is ignored, as before. Prose with no locator still does not parse. A citation that now parses but names nothing retrieved is UNRESOLVED as…
BUG-037	The date in an ISO timestamp was never read as a date	High	The ISO date pattern ended in a word boundary. In 2026-03-03T09 the day is followed by T, a word character, so the boundary never matched and the timestamp carried no date. Every log format with ISO timestamps was affected; the document corpora (dates written apart) were not.	The pattern ends "not followed by a digit" instead. A number running on (2026-03-031) is still no date. Re-run: all four findings SUPPORTED at 0.88.
BUG-038	Asked whether benign logs showed an attack, the agent described ordinary activity	High	Three correct behaviours met. BUG-037's fix grounds event dates, so the findings verified; BUG-030 keeps single-source claims when the question asks for an assessment; and the relevance gate (a model call) judged "what happened on the server" relevant to "did anything bad happen". Narrowing the…	Deterministic, once per run: when the objective asks about an attack and the documents are logs in which no ATT&CK rule flagged a single event, every candidate is discarded, with an event and the reason. Accepted cost, documented in architecture/evaluation-v2.md: an attack no rule covers reports…
BUG-039	A claim could place an event on an address its cited line does not hold	High	The specifics check grounded dates and figures, not addresses; the model verifier read the line and passed the claim.	IPv4 addresses are specifics, matched exactly: a claim cannot state an address none of its cited lines contains.
BUG-040	Tests wrote their missions into the console's run history	Medium	tests/integration/test_api_persistence.py starts real missions through the registry, and the registry writes a recording into .agent/traces/ when a run ends. No fixture redirected it.	An autouse fixture points recordings and run archives at the test's temporary folder. The 80 files already written are left for the user to remove with the history page's delete (U1b); they are hidden there by default as failed runs with no findings.
BUG-041	The frontend typed the report's limitations as text; they are objects	Medium	FinalReport.limitations is list[Limitation] (description, cause) on the backend; the hand-written frontend type said string[]. No page read the field until the journal's closing sections, which trimmed each one as a string.	The type is Limitation[]; the journal shows "description (cause)". An ErrorBoundary around every page means a failure like this now shows its message instead of a blank screen. The Phase 51 investigation report's limitations really are strings, and are unchanged.

8. Testing Tools Used

Tool	Category	How it is used
pytest	Test runner (Python)	Unit, integration, invariant and adversarial suites; markers for integration and llm tests
pytest-asyncio	Async testing	The engine, API and stores are asynchronous
pytest-cov / coverage	Coverage	Coverage report and a floor on the intelligence and schema packages in CI
pytest-randomly	Order randomisation	Reorders tests every run to expose leaked state; prints the seed
HTTPX ASGITransport	API testing	Calls the FastAPI app in-process, including SSE streams
EchoProvider	Deterministic model double	Recorded or synthesised responses; never touches the network
Vitest	Test runner (TypeScript)	Frontend logic: routes, journal, board, proof, theme, components
TypeScript compiler (tsc)	Static typing	strict, noUncheckedIndexedAccess, exactOptionalPropertyTypes
mypy --strict	Static typing (Python)	176 source modules, no errors
Ruff	Linting and formatting	Including security (S) and blind-except (BLE) rules
Alembic	Migration testing	Upgrade to head, then downgrade and upgrade the newest migration
Docker Compose / service containers	Test infrastructure	PostgreSQL 16 + pgvector and Neo4j 5 for integration tests
GitHub Actions	Continuous integration	Seven ordered jobs on every push and pull request; Pages deployment
Evaluation harness (app.cli eval)	Agent evaluation	Ten metrics over fourteen scenarios, thresholds, repeat runs
Trust, injection, challenge and ATT&CK benchmarks	Domain benchmarks	Deterministic; re-run in CI and compared with committed reports
Headless Chrome (DevTools protocol)	End-to-end UI checks	Screenshots of live runs at several widths; clicking through tabs
scripts/healthcheck.py	Environment check	Postgres, the vector extension and the configured model
OpenAPI export --check	Contract testing	Fails when the committed API contract is stale

9. Project Schedule

Work proceeded in numbered phases, one commit per phase, each closed only when its tests passed and the engineering record (.claude/) was updated. The repository holds 100 commits on main. Dates are those of the changelog.

Phase	Date	Scope	Status
Phase 0-1	23 Sep 2026	Environment, Docker, Ollama, repository skeleton, .agent and .claude	Done
Phases 2-6	23 Sep 2026	Domain schemas, LLM abstraction, event bus, intent engine	Done
Phases 7-17	23-25 Sep 2026	Planner, router, execution, reasoning, verification, gaps, replanning, relevance gate	Done
Phases 18-24	25 Sep 2026	Report synthesis, persistence, evaluation harness, API + SSE, Mission Control, replay, CI, docs	Done
Phases 25-29	3 Oct 2026	Teammate integration: trust layer, injection guard, knowledge core, Neo4j	Done
Phases 30-35	4-5 Oct 2026	Knowledge in missions, knowledge API, 3D explorer, memory, hardening, handover	Done
Phases 36-42	5-6 Oct 2026	Member 3 screens, answer evaluator, all file types, seeing and hearing, retrieval, evaluation	Done
Phases 43-44	7 Oct 2026	Evidence Locker and chain of custody; cases	Done
Demo steps 1-6	7-8 Oct 2026	Differences vs contradictions, reading every document, real file names	Done
Phases 45-51	11 Oct 2026	Statement types, confidence “why”, hypotheses, challenge, visible planning, review and audit, report	Done
Phases 52-53	12 Oct 2026	Cyber data, incident timelines, MITRE ATT&CK mapping and attack paths	Done
U1-U5	13 Oct 2026	UI redesign: run history, design system, app shell, home and composer, investigation journal	Done
Pages preview	20 Oct 2026	Static console on GitHub Pages replaying committed runs	Done
U6	20 Oct 2026	Evidence board and investigation pin board	Done
U7	20 Oct 2026	Verification receipt; comparison with the plain model	Done
U8-U11	Planned	Library, Insights, Settings, polish and first-run tour	Planned
B1-B2	Planned	Benchmarks against plain and frontier models; multimodal evaluation	Planned
B3-B5	Planned	Better reading (docling, pyannote, .evtx); independent checks (AlignScore, second reviewer); Sigma rules	Planned
B6-B10	Planned	Accounts and roles, full domain packs, held-out tuning, extras, academic deliverables	Planned

9.1 Milestones

- 25 Sep 2026 - first end-to-end investigation over HTTP with live streaming; first evaluation baseline.
- 6 Oct 2026 - fourteen-scenario baseline, run three times with identical results; teammate integration complete.
- 12 Oct 2026 - professional features complete: custody, cases, review, audit, reports, cyber.
- 20 Oct 2026 - evidence board, pin board and proof; public preview on GitHub Pages.

9.2 Remaining work

The interface phases U8 to U11 complete the redesign. Part B then strengthens the evidence that the approach works: benchmarks against the plain model and frontier models scored against known answers (FActScore and ALCE methods), multimodal evaluation with held-out scenarios, an independent non-LLM checker that can credit cross-document conclusions, and enterprise controls.

10. Graphical User Interface

The screenshots below are taken from the running console: live investigations on the local model, and the static preview. The interface is light by default with a dark theme in Settings.

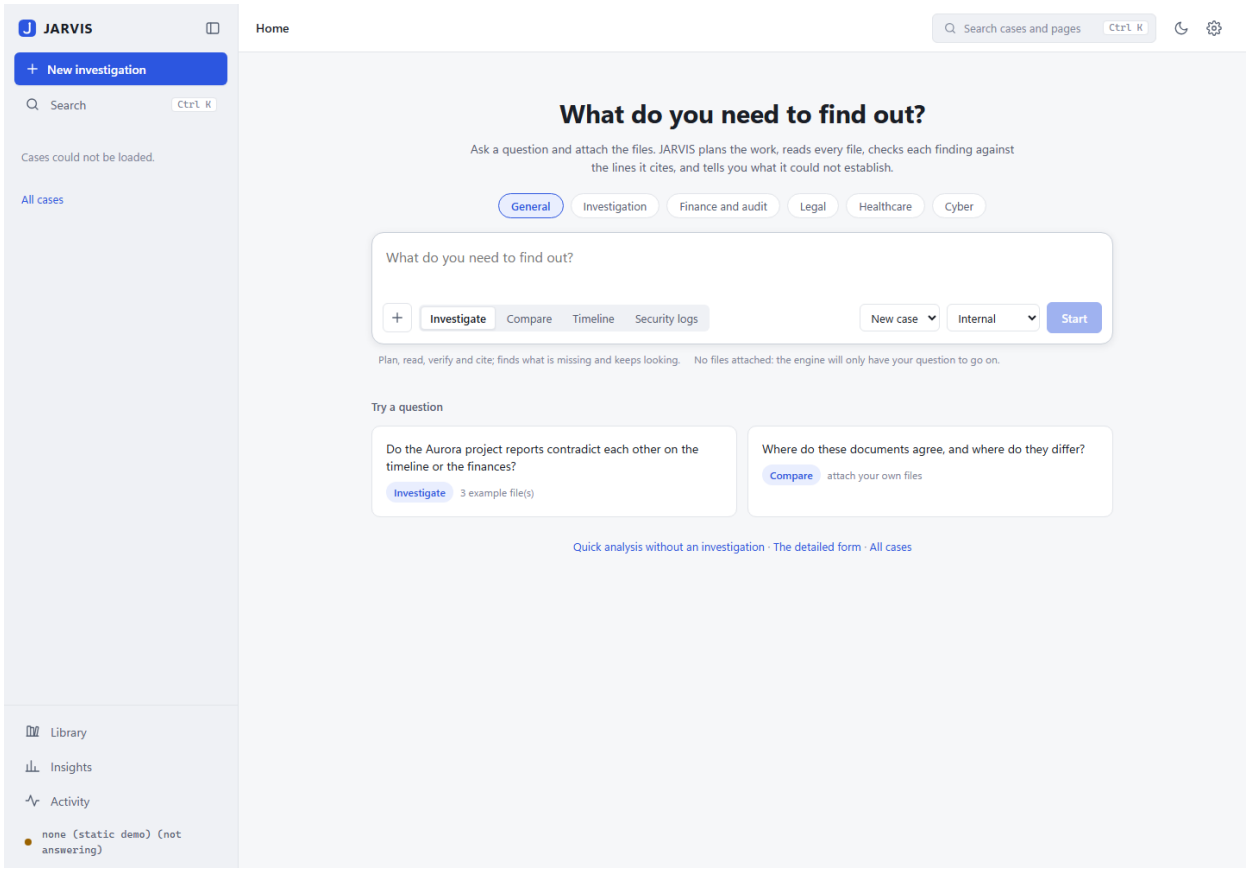


Figure 1 - Home: “What do you need to find out?”, the composer, modes and domain packs

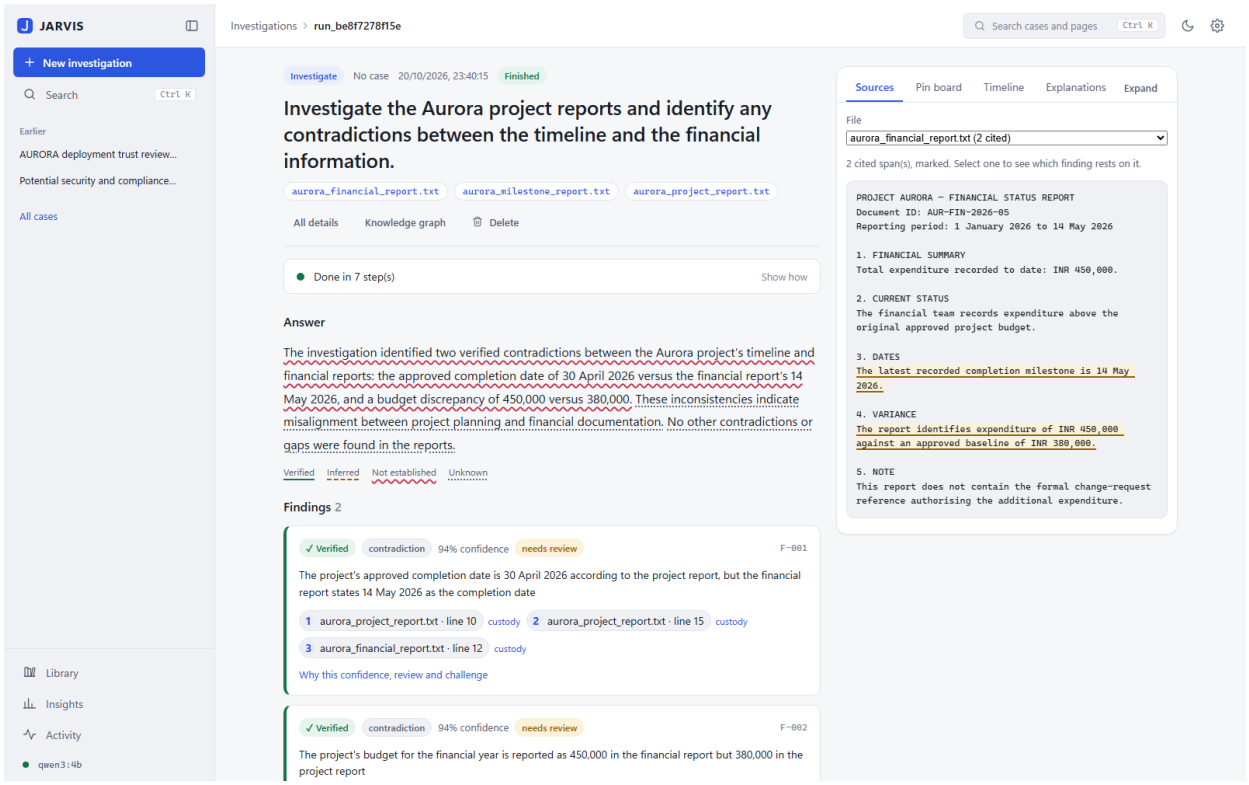


Figure 2 - Investigation journal with the evidence board: the answer marked sentence by sentence, findings with citations, and the cited spans highlighted in the source file

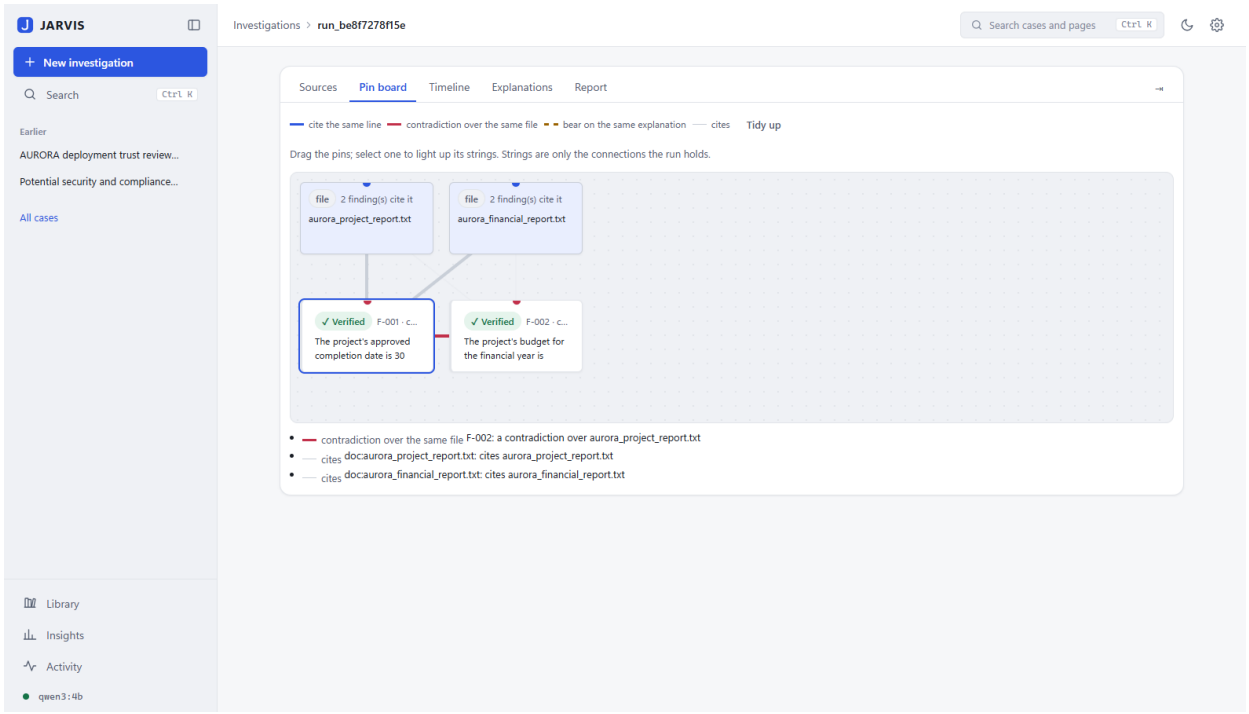


Figure 3 - The investigation pin board: files and findings joined only by connections the run holds; a contradiction string between F-001 and F-002

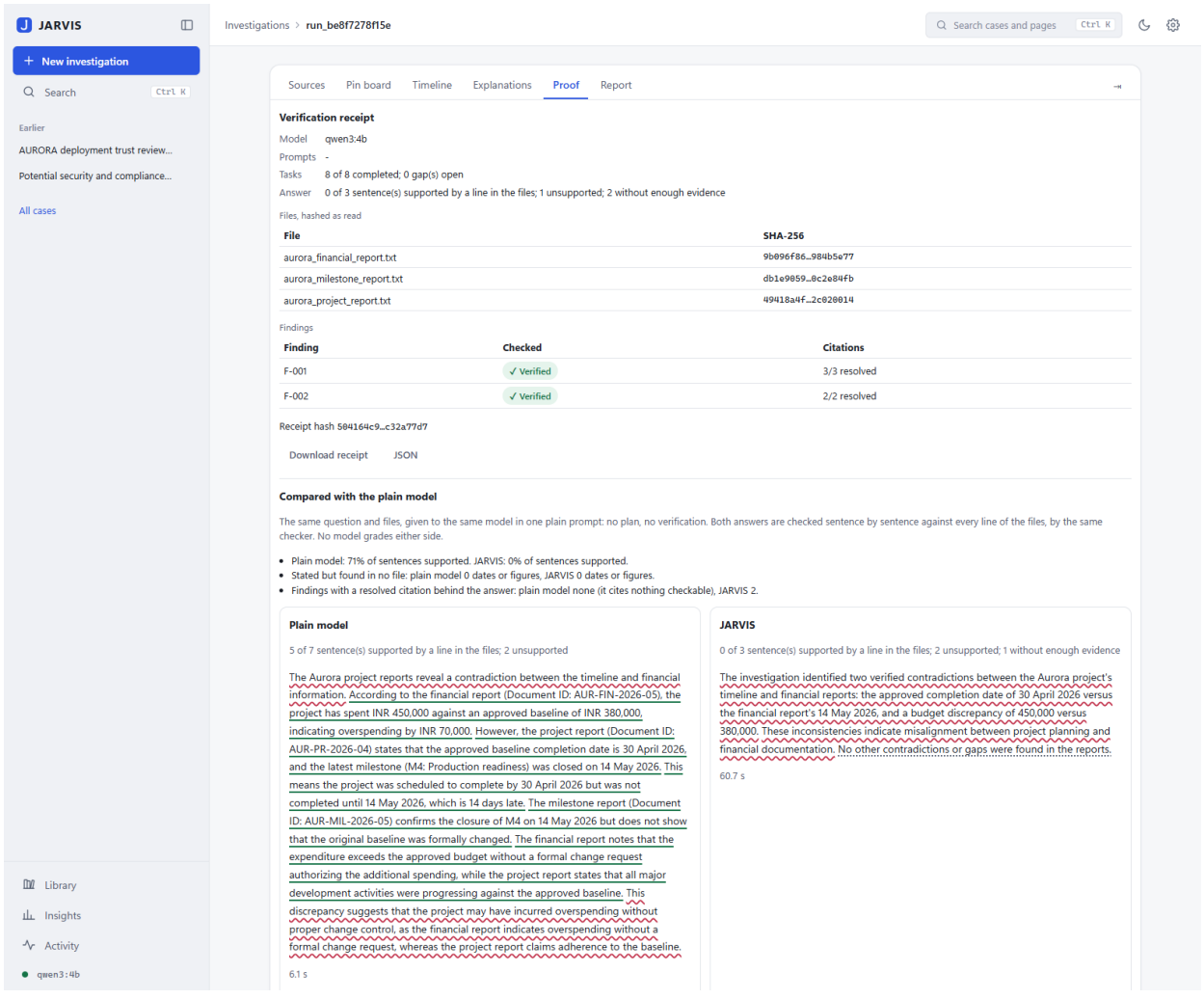


Figure 4 - Proof: the verification receipt (file hashes, findings, receipt hash) and the plain model side by side

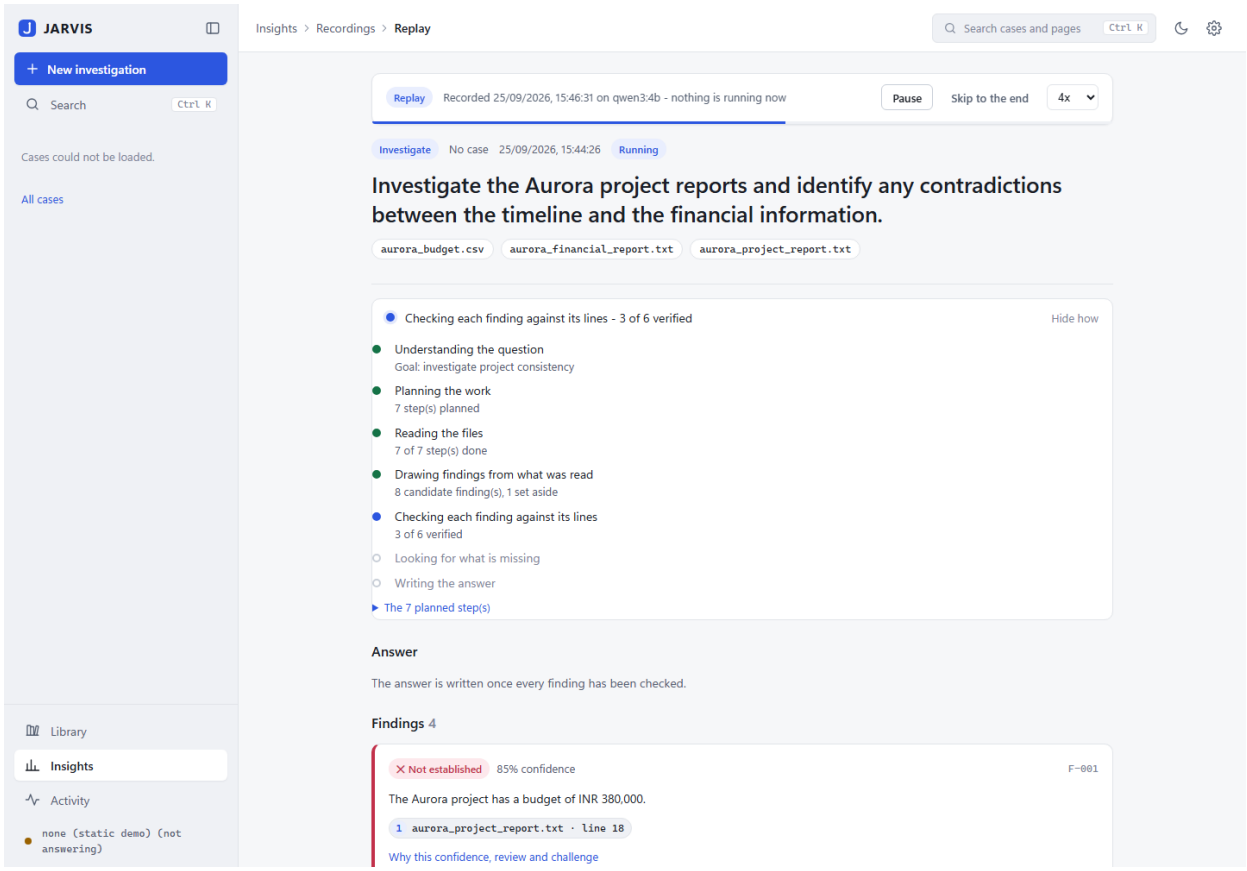


Figure 5 - Replay in the journal: a recorded run plays through the same interface and says it is a replay

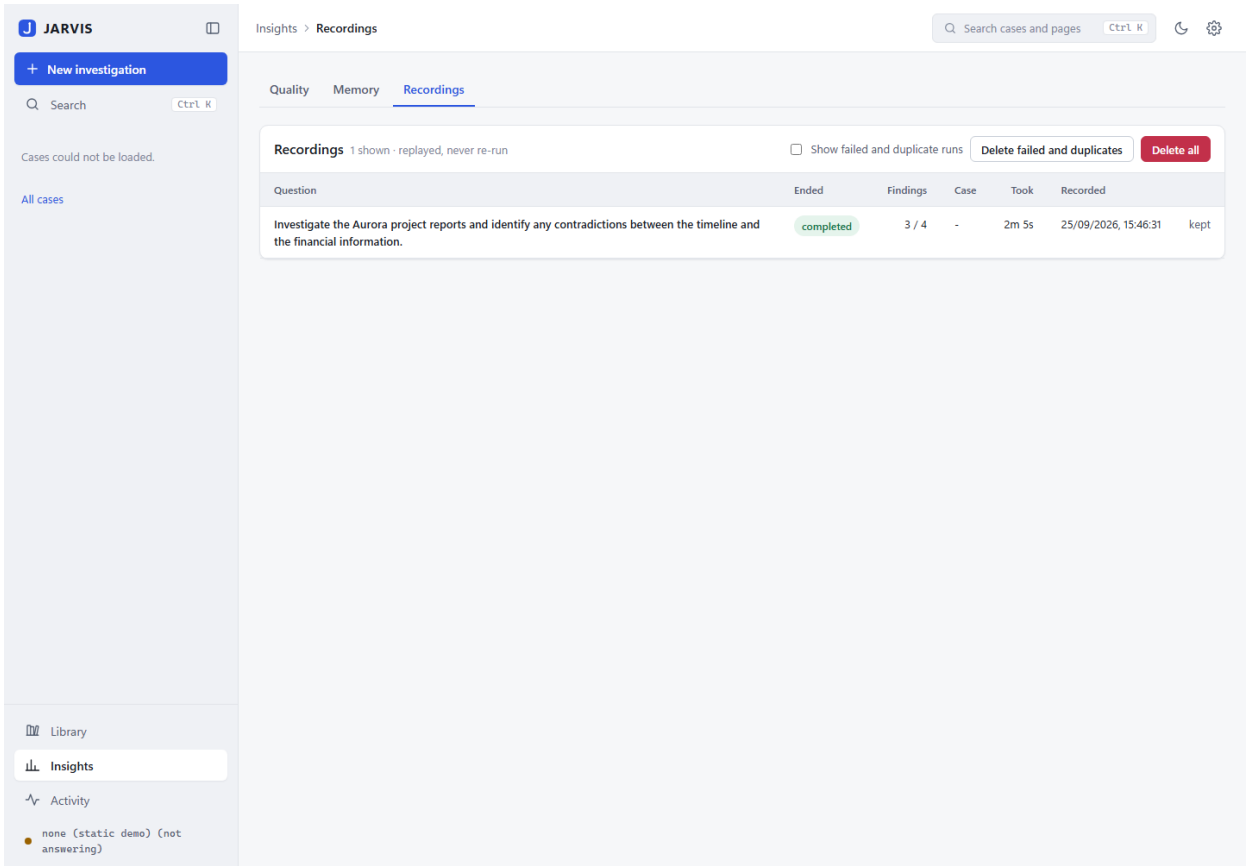


Figure 6 - Recordings as records: question, outcome, findings, duration

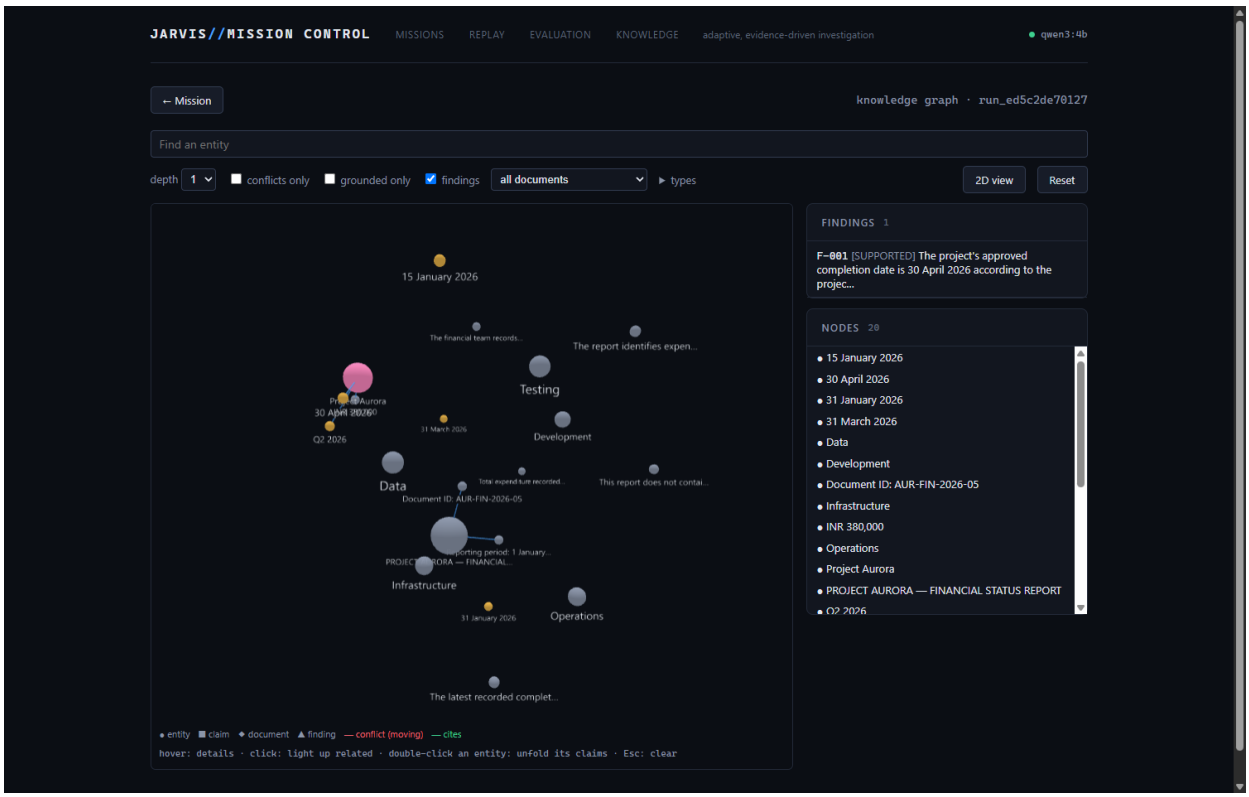


Figure 7 - The 3D knowledge graph explorer

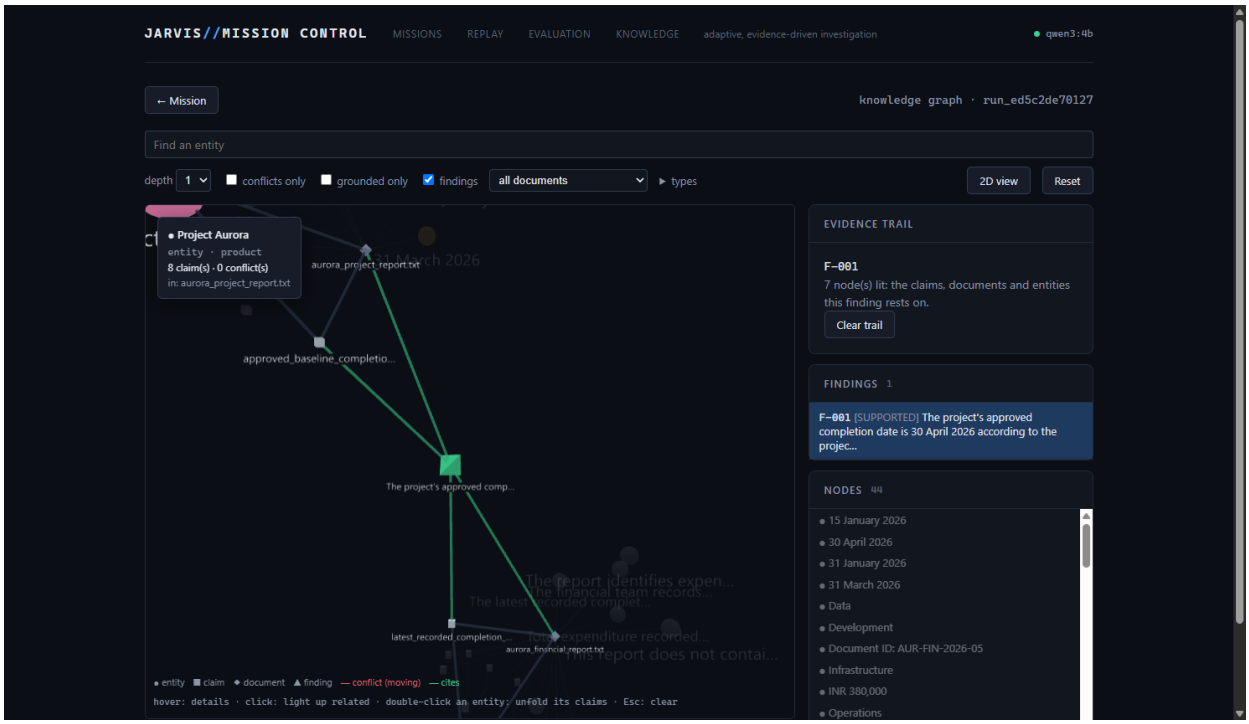


Figure 8 - A finding's trail to the evidence it rests on

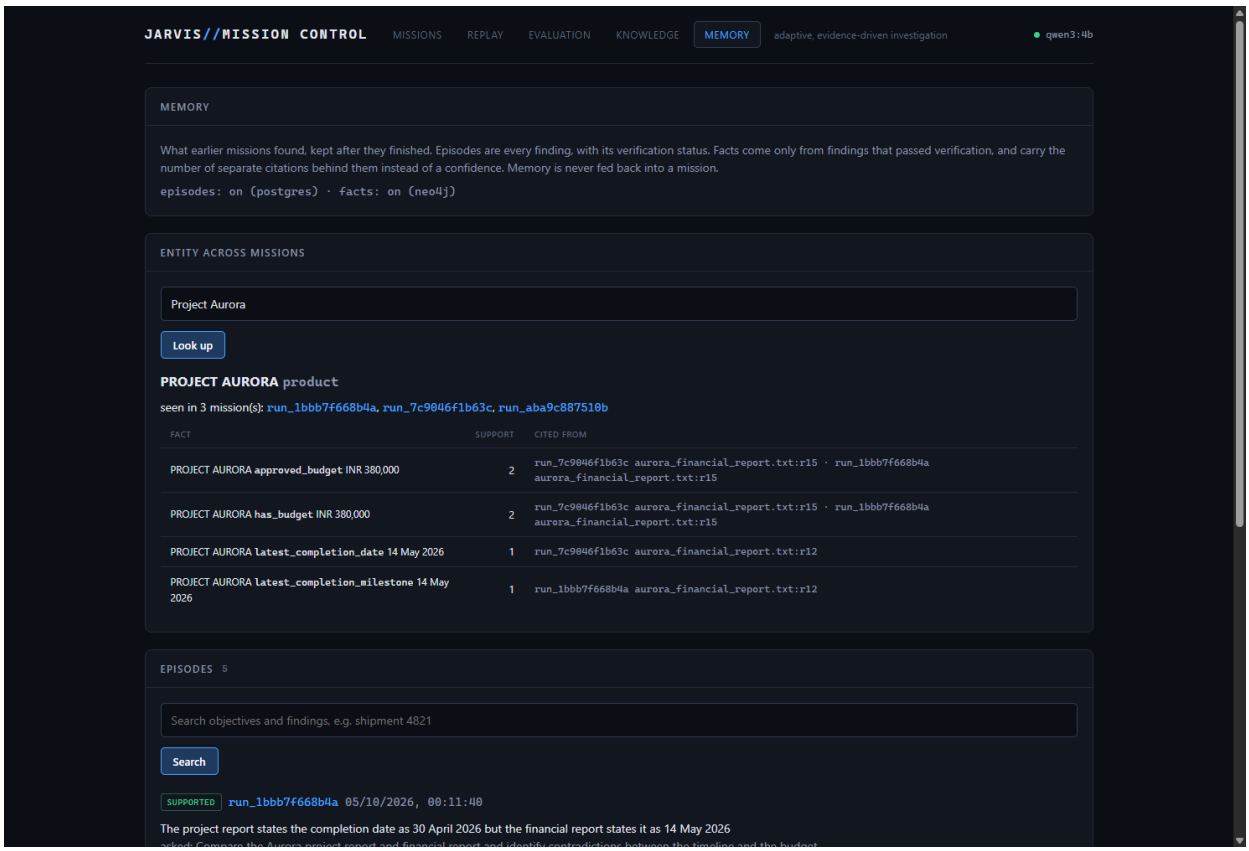


Figure 9 - Memory across investigations

11. Conclusion

JARVIS shows that a small local model can carry out document investigations that are checkable: every finding is tied to a file and a line, checked by a verifier that does not see the reasoning, and scored by a confidence the model is never asked for. The engineering record is as important as the code - forty-one defects found by running the system, each explained and fixed under a test, and an evaluation harness that refuses to flatter the agent. The remaining work is chiefly about stronger external evidence: comparison against larger models on known answers, and an independent checker able to credit conclusions that span documents.